

1. List all customers and the total number of orders they have placed. Show only customers with more than 5 orders. Sort by total orders descending.

Query:

```
SELECT c.customerid , c.companyname,  
COUNT(o.orderid) AS total_orders  
FROM customers c  
JOIN orders o  
ON c.customerid = o.customerid  
GROUP BY c.customerid, c.companyname  
HAVING COUNT(o.orderid) > 5  
ORDER BY total_orders DESC;
```

Output:

	customerid [PK] character (5)	companyname character varying (40)	total_orders bigint
1	SAVEA	Save-a-lot Markets	31
2	ERNSH	Ernst Handel	30
3	QUICK	QUICK-Stop	28
4	FOLKO	Folk och fä HB	19
5	HUNGO	Hungry Owl All-Night Grocers	19
6	HILAA	HILARION Abboton	18

2. Retrieve the total sales amount per customer by joining customers, orders, and order_details. Show only customers whose total sales exceed 10,000. Sort by total sales descending.

Query:

```
SELECT c.customerid, c.companyname,  
SUM(od.unitprice * od.quantity * (1 - discount)) AS total_sales  
FROM customers c  
JOIN orders o  
ON c.customerid = o.customerid  
JOIN order_details od  
ON od.orderid = o.orderid  
GROUP BY c.customerid, c.companyname  
HAVING SUM(od.unitprice * od.quantity * (1 - discount)) > 10000  
ORDER BY total_sales DESC;
```

Output:

	customerid [PK] character (5)	companyname character varying (40)	total_sales double precision
1	QUICK	QUICK-Stop	110277.30488350891
2	ERNSH	Ernst Handel	104874.97835350106
3	SAVEA	Save-a-lot Markets	104361.94986812412
4	RATTC	Rattlesnake Canyon Grocery	51097.80048773782
5	HUNGO	Hungry Owl All-Night Groc...	49979.90494634077

3. Get the number of products per category. Show only categories having more than 10 products. Sort by product count descending.

Query:

```
SELECT c.categoryid, c.categoryname,
COUNT(p.productid) as total_products
FROM categories c
JOIN products p
ON c.categoryid = p.categoryid
GROUP BY c.categoryid, c.categoryname
HAVING COUNT(p.productid) > 10
ORDER BY total_products DESC;
```

Output:

	categoryid [PK] integer	categoryname character varying (15)	total_products bigint
1	3	Confections	13
2	2	Condiments	12
3	1	Beverages	12
4	8	Seafood	12

4. Display the total quantity sold per product. Include only products where total quantity sold is greater than 100. Sort by quantity descending.

Query:

```
SELECT p.productid, p.productname,
SUM(od.quantity) AS total_quantity
FROM order_details od
JOIN products p
ON od.productid = p.productid
GROUP BY p.productid, p.productname
HAVING SUM(od.quantity) > 100
ORDER BY total_quantity DESC;
```

Output:

	productid [PK] integer	productname character varying (40)	total_quantity bigint
1	60	Camembert Pierrot	1577
2	59	Raclette Courdavault	1496
3	31	Gorgonzola Telino	1397
4	56	Gnocchi di nonna Alice	1263
5	16	Pavlova	1158

5. Find the total number of orders handled by each employee. Show only employees who handled more than 20 orders. Sort by order count descending.

Query:

```
SELECT o.employeeid,  
CONCAT(e.firstname, ' ', e.lastname) AS name,  
COUNT(o.orderid) AS total_orders  
FROM orders o  
JOIN employees e  
ON o.employeeid = e.employeeid  
GROUP BY o.employeeid, e.firstname, e.lastname  
HAVING COUNT(o.orderid) > 20  
ORDER BY total_orders DESC;
```

Output:

	employeeid integer	name text	total_orders bigint
1	4	Margaret Peaco...	156
2	3	Janet Leverling	127
3	1	Nancy Davolio	123
4	8	Laura Callahan	104
5	2	Andrew Fuller	96

6. Retrieve the total sales per category by joining categories, products, and order_details. Show only categories with total sales above 50,000. Sort by total sales descending.

Query:

```
SELECT c.categoryid , c.categoryname,  
SUM(od.unitprice * od.quantity * (1 - discount)) as total_sales
```

```

FROM orders o
JOIN order_details od
ON o.orderid = od.orderid
JOIN products p
ON p.productid = od.productid
JOIN categories c
ON c.categoryid = p.categoryid
GROUP BY c.categoryid
HAVING SUM(od.unitprice * od.quantity * (1 - discount)) > 50000
ORDER BY total_sales DESC;

```

Output:

	categoryid [PK] integer	categoryname character varying (15)	total_sales double precision
1	1	Beverages	267868.1797437187
2	4	Dairy Products	234507.2847417593
3	3	Confections	167357.22484282582
4	6	Meat/Poultry	163022.35939204693
5	8	Seafood	131261.73734469974

7. List suppliers and the number of products they supply. Show only suppliers who supply more than 5 products. Sort by product count descending.

Query:

```

SELECT s.supplierid, s.companyname,
COUNT(p.productid) AS total_products
FROM products p
JOIN suppliers s
ON p.supplierid = s.supplierid
GROUP BY s.supplierid , s.companyname
HAVING COUNT(p.productid) > 5
ORDER BY total_products DESC;

```

Output: - no records found

	supplierid [PK] integer	companyname character varying (40)	total_products bigint

8. Get the average unit price per category. Show only categories where the average price is above 30. Sort by average price descending.

Query:

```
SELECT c.categoryid, c.categoryname,  
AVG(p.unitprice) AS avg_price  
FROM products p  
JOIN categories c  
ON p.categoryid = c.categoryid  
GROUP BY c.categoryid, c.categoryname  
HAVING AVG(p.unitprice) > 30  
ORDER BY avg_price DESC;
```

Output:

	categoryid [PK] integer	categoryname character varying (15)	avg_price numeric
1	6	Meat/Poultry	54.006666666666667
2	1	Beverages	37.979166666666667
3	7	Produce	32.370000000000000

9. Display the total revenue generated per employee (orders + order_details). Show only employees generating more than 20,000 in revenue. Sort by revenue descending.

Query:

```
SELECT e.employeeid,  
CONCAT(e.firstname, ' ', e.lastname) AS name,  
SUM(od.unitprice * od.quantity * (1 - od.discount)) AS total_revenue  
FROM orders o  
JOIN order_details od  
ON o.orderid = od.orderid  
JOIN employees e  
ON e.employeeid = o.employeeid  
GROUP BY e.employeeid, e.firstname, e.lastname  
HAVING SUM(od.unitprice * od.quantity * (1 - od.discount)) > 20000  
ORDER BY total_revenue DESC;
```

Output:

	employeeid [PK] integer	name text	total_revenue double precision
1	4	Margaret Peaco...	232890.84573612877
2	3	Janet Leverling	202812.84285137767
3	1	Nancy Davolio	192107.60437067723
4	2	Andrew Fuller	166537.75483498262
5	8	Laura Callahan	126862.2774064841

10. Retrieve the number of orders shipped to each country. Show only countries with more than 10 orders. Sort by order count descending.

Query:

```
SELECT o.shipcountry,
COUNT(o.orderid) AS total_orders
FROM orders o
GROUP BY o.shipcountry
HAVING COUNT(o.orderid) > 10
ORDER BY total_orders DESC;
```

Output:

	shipcountry character varying (15)	total_orders bigint
1	USA	122
2	Germany	122
3	Brazil	83
4	France	77
5	UK	56

11. Find customers and the average order value (orders + order_details). Show only customers with average order value greater than 500. Sort by average descending.

Query:

```
WITH order_total_details AS (
    SELECT o.customerid, o.orderid,
    SUM(od.unitprice * od.quantity * (1 - od.discount)) AS order_total
    FROM orders o
    JOIN order_details od
    ON o.orderid = od.orderid
    GROUP BY o.orderid, o.customerid
    -- summing up all the individual items in an order
```

```

)
SELECT c.customerid, c.companyname,
AVG(otd.order_total) AS avg_order_value
FROM customers c
JOIN order_total_details otd
ON c.customerid = otd.customerid
GROUP BY c.customerid, c.companyname
HAVING AVG(otd.order_total) > 500
ORDER BY avg_order_value DESC;

```

Output:

	customerid [PK] character (5) 	companyname character varying (40) 	avg_order_value double precision 
1	QUICK	QUICK-Stop	3938.4751744110326
2	ERNSH	Ernst Handel	3495.83261178337
3	SAVEA	Save-a-lot Markets	3366.514511874972
4	RATTC	Rattlesnake Canyon Grocery	2838.766693763212
5	HUNGO	Hungry Owl All-Night Grocers	2630.5213129653043

12. Get the top-selling products per category (by total quantity sold). Show only products with total quantity sold above 200. Sort within category by quantity descending.

Query:

```

SELECT c.categoryid, c.categoryname, p.productid, p.productname,
SUM(od.quantity) AS total_quantity
FROM products p
JOIN categories c
ON p.categoryid = c.categoryid
JOIN order_details od
ON p.productid = od.productid
GROUP BY c.categoryid, c.categoryname, p.productid, p.productname
HAVING SUM(od.quantity) > 200
ORDER BY c.categoryid, total_quantity DESC;

```

Output:

	categoryid integer	categoryname character varying (15)	productid integer	productname character varying (40)	total_quantity bigint
1	1	Beverages	75	Rhönbräu Klosterbier	1155
2	1	Beverages	24	Guaraná Fantástica	1125
3	1	Beverages	2	Chang	1057
4	1	Beverages	76	Lakkalikööri	981
5	1	Beverages	35	Steeleye Stout	883

13. Retrieve the total discount given per product (order_details). Show only products where total discount exceeds 1,000. Sort by discount descending.

Query:

```
SELECT p.productid, p.productname,
SUM(od.unitprice * od.quantity * od.discount) AS total_discount
FROM products p
JOIN order_details od
ON p.productid = od.productid
GROUP BY p.productid
HAVING SUM(od.unitprice * od.quantity * od.discount) > 1000
ORDER BY total_discount DESC;
```

Output:

	productid [PK] integer	productname character varying (40)	total_discount double precision
1	38	Côte de Blaye	8587.465096551552
2	29	Thüringer Rostbratwurst	7367.728025485949
3	59	Raclette Courdavault	5140.300090570003
4	60	Camembert Pierrot	3460.520046980679
5	51	Manjimup Dried Apples	2922.9500460036097

14. List employees and the number of unique customers they handled. Show only employees who handled more than 15 unique customers. Sort by count descending.

Query:

```
SELECT e.employeeid,
CONCAT(e.firstname, ' ', e.lastname) AS name,
COUNT(DISTINCT o.customerid) AS unique_orders
FROM employees e
JOIN orders o
```



```

ON e.employeeid = o.employeeid
GROUP BY e.employeeid, e.firstname, e.lastname
HAVING COUNT(DISTINCT o.customerid) > 15
ORDER BY unique_orders DESC;

```

Output:

	employeeid [PK] integer	name text	unique_orders bigint
1	4	Margaret Peaco...	75
2	1	Nancy Davolio	65
3	3	Janet Leverling	63
4	2	Andrew Fuller	59
5	8	Laura Callahan	56

15. Find the monthly total sales (year + month) using orders and order_details. Show only months where total sales exceed 30,000. Sort by year and month ascending.

Query:

```

SELECT TO_CHAR(o.orderdate, 'YYYY-MM') AS year_month,
SUM(od.unitprice * od.quantity * (1 - od.discount)) AS total_sales
FROM orders o
JOIN order_details od
ON o.orderid = od.orderid
GROUP BY TO_CHAR(o.orderdate, 'YYYY-MM')
HAVING SUM(od.unitprice * od.quantity * (1 - od.discount)) > 30000
ORDER BY year_month;

```

-- instead of TO_CHAR() and converting orderdate as a string,
-- DATE_TRUNC("month", orderdate) can be used, it defaults to anything smaller than a month from the timestamp
-- TO_CHAR() is used then to format as string.
-- NOTE: Query 1 groups by formatted string, Query 2 groups by actual date (more reliable).

```

SELECT DATE_TRUNC('month', o.orderdate) AS year_month,
SUM(od.unitprice * od.quantity * (1 - od.discount)) AS total_sales
FROM orders o
JOIN order_details od
ON o.orderid = od.orderid
GROUP BY DATE_TRUNC('month', o.orderdate)
HAVING SUM(od.unitprice * od.quantity * (1 - od.discount)) > 30000
ORDER BY year_month;

```

Output:

	year_month timestamp without time zone 	total_sales double precision 
1	1996-10-01 00:00:00	37515.72491448447
2	1996-11-01 00:00:00	45600.04494312592
3	1996-12-01 00:00:00	45239.629965431996
4	1997-01-01 00:00:00	61258.0699304849
5	1997-02-01 00:00:00	38483.63494294286

TRANSACTIONS AND STORED PROCEDURES

1. Create a stored procedure to insert a new customer. Use a transaction so that if any required value is missing, the insert is rolled back.

```
CREATE OR REPLACE PROCEDURE proc_insert_customer(  
    p_customer_id CHAR(5),  
    p_company_name VARCHAR(40),  
    p_contact_name VARCHAR(30),  
    p_contact_title VARCHAR(30) DEFAULT NULL,  
    p_address VARCHAR(60) DEFAULT NULL,  
    p_city VARCHAR(15) DEFAULT NULL,  
    p_region VARCHAR(15) DEFAULT NULL,  
    p_postal_code VARCHAR(10) DEFAULT NULL,  
    p_country VARCHAR(15) DEFAULT NULL,  
    p_phone VARCHAR(24) DEFAULT NULL,  
    p_fax VARCHAR(24) DEFAULT NULL  
)  
LANGUAGE plpgsql  
AS $$  
BEGIN  
    IF p_customer_id IS NULL OR TRIM(p_customer_id) = '' THEN  
        RAISE EXCEPTION 'CustomerID is required';  
    END IF;  
  
    IF p_company_name IS NULL OR TRIM(p_company_name) = '' THEN  
        RAISE EXCEPTION 'CompanyName is required';  
    END IF;  
  
    IF p_contact_name IS NULL OR TRIM(p_contact_name) = '' THEN
```

```

        RAISE EXCEPTION 'ContactName is required';
    END IF;

    INSERT INTO customers (
        customerid, companyname, contactname,
        contacttitle, address, city,
        region, postalcode, country,
        phone, fax
    )
    VALUES (
        p_customer_id, p_company_name, p_contact_name,
        p_contact_title, p_address, p_city,
        p_region, p_postal_code, p_country,
        p_phone, p_fax
    );

END;
$$;

CALL proc_insert_customer ('AVISH', 'Hsivad Tech', 'Davish');

```

2. Create a stored procedure to place a new order for an existing customer with one product. Insert into orders and order_details in a single transaction.

Procedure:

```

CREATE OR REPLACE PROCEDURE proc_place_new_order(
    p_customer_id CHAR(5),
    p_employee_id INT,
    p_product_id INT,
    p_quantity SMALLINT,
    p_unit_price NUMERIC(10,2),
    p_discount REAL DEFAULT 0
)
LANGUAGE plpgsql
AS $$
DECLARE
    v_order_id INT;
    v_stock SMALLINT;
    v_db_price NUMERIC(10,2);
    v_discontinued INT;
BEGIN

```

```

-- validate customer exists
IF NOT EXISTS (
    SELECT 1 FROM customers WHERE customerid = p_customer_id
) THEN
    RAISE EXCEPTION 'customer % does not exist', p_customer_id;
END IF;

-- validate employee exists
IF NOT EXISTS (
    SELECT 1 FROM employees WHERE employeeid = p_employee_id
) THEN
    RAISE EXCEPTION 'employee % does not exist', p_employee_id;
END IF;

-- validate product exists and fetch stock, price and discontinued status
SELECT unitsinstock, unitprice, discontinued
INTO v_stock, v_db_price, v_discontinued
FROM products
WHERE productid = p_product_id;

IF NOT FOUND THEN
    RAISE EXCEPTION 'product % does not exist', p_product_id;
END IF;

-- ensure product is not discontinued
IF v_discontinued = 1 THEN
    RAISE EXCEPTION 'product % is discontinued', p_product_id;
END IF;

-- validate quantity
IF p_quantity <= 0 THEN
    RAISE EXCEPTION 'quantity must be greater than 0';
END IF;

-- validate sufficient stock
IF v_stock < p_quantity THEN
    RAISE EXCEPTION 'insufficient stock for product % (available: %, requested: %)',
        p_product_id, v_stock, p_quantity;
END IF;

-- validate discount range
IF p_discount < 0 OR p_discount > 1 THEN
    RAISE EXCEPTION 'discount must be between 0 and 1';
END IF;

```

```
-- optionally enforce price consistency (use DB price if mismatch)
IF p_unit_price <> v_db_price THEN
    RAISE NOTICE 'unit price overridden with current product price';
    p_unit_price := v_db_price;
END IF;
```

```
-- insert into orders with all columns
```

```
INSERT INTO orders (
    customerid,
    employeeid,
    orderdate,
    requireddate,
    shippeddate,
    shipvia,
    freight,
    shipname,
    shipaddress,
    shipcity,
    shipregion,
    shippostalcode,
    shipcountry
)
VALUES (
    p_customer_id,
    p_employee_id,
    CURRENT_DATE,
    CURRENT_DATE + INTERVAL '7 days',
    NULL,
    1,
    0,
    'Default Ship Name',
    'Default Address',
    'Default City',
    NULL,
    '000000',
    'Default Country'
)
RETURNING orderid INTO v_order_id;
```

```
-- insert into order_details with all columns
```

```
INSERT INTO order_details (
    orderid,
    productid,
```

```

        unitprice,
        quantity,
        discount
    )
VALUES (
    v_order_id,
    p_product_id,
    p_unit_price,
    p_quantity,
    p_discount
);

-- update stock after successful order placement
UPDATE products
SET unitsinstock = unitsinstock - p_quantity
WHERE productid = p_product_id;

EXCEPTION
WHEN OTHERS THEN
    -- rollback handled automatically on failure
    RAISE;
END;
$$;

```

Output : Procedure doing business logic validation and throws an error.

```

ERROR: insufficient stock for product 3 (available: 13, requested: 20)
CONTEXT: PL/pgSQL function proc_place_new_order(character,integer,integer,smallint,numeric,real) line 44 at RAISE
SQL state: P0001

```

3. Create a stored procedure to update product stock after an order is placed. If stock is not enough, rollback the transaction.

Procedure:

```

CREATE OR REPLACE PROCEDURE proc_update_product_stock(
    p_product_id INT,
    p_quantity SMALLINT
)
LANGUAGE plpgsql
AS $$
DECLARE
    v_stock SMALLINT;

```

```

BEGIN
    -- fetch stock with row lock to avoid race conditions
    SELECT unitsinstock
    INTO v_stock
    FROM products
    WHERE productid = p_product_id
    FOR UPDATE;

    IF NOT FOUND THEN
        RAISE EXCEPTION 'product % does not exist', p_product_id;
    END IF;

    -- validate quantity
    IF p_quantity <= 0 THEN
        RAISE EXCEPTION 'quantity must be greater than 0';
    END IF;

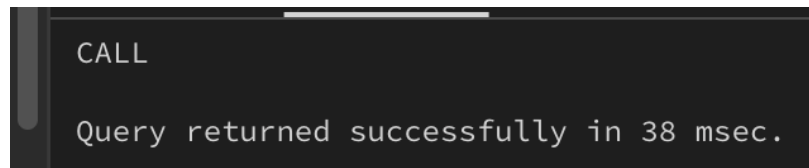
    -- check stock availability
    IF v_stock < p_quantity THEN
        RAISE EXCEPTION 'insufficient stock for product % (available: %, requested: %)',
            p_product_id, v_stock, p_quantity;
    END IF;

    -- update stock
    UPDATE products
    SET unitsinstock = unitsinstock - p_quantity
    WHERE productid = p_product_id;

END;
$$;

```

Output:



```

CALL

Query returned successfully in 38 msec.

```

4. Create a stored procedure to cancel an order. Delete records from order_details first, then from orders, using a transaction.

Procedure:

```

CREATE OR REPLACE PROCEDURE proc_cancel_order(
    p_order_id INT
)

```

```

LANGUAGE plpgsql
AS $$
BEGIN
    -- validate order exists
    IF NOT EXISTS (
        SELECT 1 FROM orders WHERE orderid = p_order_id
    ) THEN
        RAISE EXCEPTION 'order % does not exist', p_order_id;
    END IF;

    -- delete dependent records from order_details first (to maintain FK integrity)
    DELETE FROM order_details
    WHERE orderid = p_order_id;

    -- delete the order from orders table
    DELETE FROM orders
    WHERE orderid = p_order_id;

END;
$$;

-- valid cancellation
CALL proc_cancel_order(10248);

```

Output:

```

CALL

Query returned successfully in 49 msec.

```

5. Create a stored procedure to transfer products from one supplier to another. If the old supplier or new supplier does not exist, rollback.

Procedure:

```

CREATE OR REPLACE PROCEDURE proc_transfer_products_supplier(
    p_old_supplier_id INT,
    p_new_supplier_id INT
)
LANGUAGE plpgsql
AS $$
DECLARE
    v_count INT;
BEGIN

```



```

-- validate old supplier exists
IF NOT EXISTS (
    SELECT 1 FROM suppliers WHERE supplierid = p_old_supplier_id
) THEN
    RAISE EXCEPTION 'old supplier % does not exist', p_old_supplier_id;
END IF;

-- validate new supplier exists
IF NOT EXISTS (
    SELECT 1 FROM suppliers WHERE supplierid = p_new_supplier_id
) THEN
    RAISE EXCEPTION 'new supplier % does not exist', p_new_supplier_id;
END IF;

-- prevent same supplier transfer
IF p_old_supplier_id = p_new_supplier_id THEN
    RAISE EXCEPTION 'both suppliers cannot be the same';
END IF;

-- update products to new supplier
UPDATE products
SET supplierid = p_new_supplier_id
WHERE supplierid = p_old_supplier_id;

-- check if any products were actually transferred
GET DIAGNOSTICS v_count = ROW_COUNT;

IF v_count = 0 THEN
    RAISE NOTICE 'no products found for supplier %', p_old_supplier_id;
END IF;

END;
$$;

CALL proc_transfer_products_supplier(1, 2);

```

Output:

```

CALL
Query returned successfully in 72 msec.

```

6. Create a stored procedure to update the price of all products in a category by a percentage. Rollback if the percentage is less than or equal to zero.

Procedure:

```
CREATE OR REPLACE PROCEDURE proc_update_category_price(
  p_category_id INT,
  p_percentage NUMERIC
)
LANGUAGE plpgsql
AS $$
DECLARE
  v_count INT;
BEGIN
  -- validate category exists
  IF NOT EXISTS (
    SELECT 1 FROM categories WHERE categoryid = p_category_id
  ) THEN
    RAISE EXCEPTION 'category % does not exist', p_category_id;
  END IF;

  -- validate percentage (> 0 required)
  IF p_percentage <= 0 THEN
    RAISE EXCEPTION 'percentage must be greater than 0';
  END IF;

  -- update product prices using percentage increase
  UPDATE products
  SET unitprice = unitprice + (unitprice * p_percentage / 100)
  WHERE categoryid = p_category_id;

  -- check if any rows were updated
  GET DIAGNOSTICS v_count = ROW_COUNT;

  IF v_count = 0 THEN
    RAISE NOTICE 'no products found for category %', p_category_id;
  END IF;

END;
$$;

-- increase prices by 10% for category 1
CALL proc_update_category_price(1, 10);
```

Output

```
CALL
```

```
Query returned successfully in 37 msec.
```

**7. Create a stored procedure to add a new product under an existing category and supplier.
Rollback if the category or supplier does not exist.**

Procedure:

```
CREATE OR REPLACE PROCEDURE proc_add_product(
  p_product_name  VARCHAR(40),
  p_supplier_id   INT,
  p_category_id   INT,
  p_quantity_per_unit VARCHAR(20),
  p_unit_price    NUMERIC(10,2),
  p_units_in_stock SMALLINT,
  p_units_on_order SMALLINT,
  p_reorder_level SMALLINT,
  p_discontinued  INT DEFAULT 0
)
LANGUAGE plpgsql
AS $$
DECLARE
  v_product_id INT;
BEGIN
  -- validate supplier exists
  IF NOT EXISTS (
    SELECT 1 FROM suppliers WHERE supplierid = p_supplier_id
  ) THEN
    RAISE EXCEPTION 'supplier % does not exist', p_supplier_id;
  END IF;

  -- validate category exists
  IF NOT EXISTS (
    SELECT 1 FROM categories WHERE categoryid = p_category_id
  ) THEN
    RAISE EXCEPTION 'category % does not exist', p_category_id;
  END IF;

  -- validate product name
  IF p_product_name IS NULL OR LENGTH(TRIM(p_product_name)) = 0 THEN
    RAISE EXCEPTION 'product name cannot be empty';
```

```

END IF;

-- validate price
IF p_unit_price < 0 THEN
    RAISE EXCEPTION 'unit price cannot be negative';
END IF;

-- insert new product with all columns
INSERT INTO products (
    productname,
    supplierid,
    categoryid,
    quantityperunit,
    unitprice,
    unitsinstock,
    unitsonorder,
    reorderlevel,
    discontinued
)
VALUES (
    p_product_name,
    p_supplier_id,
    p_category_id,
    p_quantity_per_unit,
    p_unit_price,
    p_units_in_stock,
    p_units_on_order,
    p_reorder_level,
    p_discontinued
)
RETURNING productid INTO v_product_id;

END;
$$;

-- add a new product
CALL proc_add_product(
    'New Choco Bar',
    1,
    1,
    '10 boxes x 20 bars',
    25.50,
    100,
    0,

```

```
10,  
0  
);
```

Output:

```
CALL  
  
Query returned successfully in 36 msec.
```

8. Create a stored procedure to delete a customer only if the customer has no orders. Use a transaction and rollback if orders exist.

Procedure:

```
CREATE OR REPLACE PROCEDURE proc_delete_customer_if_no_orders(  
    p_customer_id CHAR(5)  
)  
LANGUAGE plpgsql  
AS $$  
BEGIN  
    -- validate customer exists  
    IF NOT EXISTS (  
        SELECT 1 FROM customers WHERE customerid = p_customer_id  
    ) THEN  
        RAISE EXCEPTION 'customer % does not exist', p_customer_id;  
    END IF;  
  
    -- check if customer has any orders  
    IF EXISTS (  
        SELECT 1 FROM orders WHERE customerid = p_customer_id  
    ) THEN  
        RAISE EXCEPTION 'cannot delete customer % because orders exist', p_customer_id;  
    END IF;  
  
    -- delete customer (safe since no dependent orders)  
    DELETE FROM customers  
    WHERE customerid = p_customer_id;  
  
END;  
$$;  
  
-- successful deletion (only if no orders exist)
```

```
CALL proc_delete_customer_if_no_orders('FISSA');
```

Output:

```
CALL  
  
Query returned successfully in 41 msec.
```

9. Create a stored procedure to apply a discount to all order details for a specific order. Rollback if the order does not exist.

Procedure:

```
CREATE OR REPLACE PROCEDURE proc_apply_discount_to_order(  
    p_order_id INT,  
    p_discount REAL  
)  
LANGUAGE plpgsql  
AS $$  
DECLARE  
    v_count INT;  
BEGIN  
    -- validate order exists  
    IF NOT EXISTS (  
        SELECT 1 FROM orders WHERE orderid = p_order_id  
    ) THEN  
        RAISE EXCEPTION 'order % does not exist', p_order_id;  
    END IF;  
  
    -- validate discount range (0 to 1)  
    IF p_discount < 0 OR p_discount > 1 THEN  
        RAISE EXCEPTION 'discount must be between 0 and 1';  
    END IF;  
  
    -- update all order_details for the given order  
    UPDATE order_details  
    SET discount = p_discount  
    WHERE orderid = p_order_id;  
  
    -- check if any rows were updated  
    GET DIAGNOSTICS v_count = ROW_COUNT;
```

```

IF v_count = 0 THEN
    RAISE NOTICE 'no order details found for order %', p_order_id;
END IF;

END;
$$;

-- apply 10% discount
CALL proc_apply_discount_to_order(10248, 0.1);

```

Output: Business logic inside procedure is raising error since orderid 10248 is unavailable

Data Output	Messages	Notifications
ERROR: order 10248 does not exist CONTEXT: PL/pgSQL function proc_apply_discount_to_order(integer,real) line 9 at RAISE SQL state: P0001		

10. Create a stored procedure to place an order with multiple products. Insert the order and all order items in one transaction. If any product is invalid or stock is insufficient, rollback the complete order.

Procedure:

```

CREATE OR REPLACE PROCEDURE proc_place_multi_product_order(
    p_customer_id CHAR(5),
    p_employee_id INT,
    p_items JSON -- [{"product_id":1,"quantity":5}, ...]
)
LANGUAGE plpgsql
AS $$
DECLARE
    v_order_id INT;
    v_item JSON;
    v_product_id INT;
    v_quantity SMALLINT;
    v_stock SMALLINT;
    v_price NUMERIC(10,2);
BEGIN
    -- validate customer exists
    IF NOT EXISTS (
        SELECT 1 FROM customers WHERE customerid = p_customer_id
    ) THEN
        RAISE EXCEPTION 'customer % does not exist', p_customer_id;
    END IF;

```

```

END IF;

-- validate employee exists
IF NOT EXISTS (
    SELECT 1 FROM employees WHERE employeeid = p_employee_id
) THEN
    RAISE EXCEPTION 'employee % does not exist', p_employee_id;
END IF;

-- insert order
INSERT INTO orders (
    customerid,
    employeeid,
    orderdate,
    requireddate,
    shippeddate,
    shipvia,
    freight,
    shipname,
    shipaddress,
    shipcity,
    shipregion,
    shippostalcode,
    shipcountry
)
VALUES (
    p_customer_id,
    p_employee_id,
    CURRENT_DATE,
    CURRENT_DATE + INTERVAL '7 days',
    NULL,
    1,
    0,
    'Default Ship Name',
    'Default Address',
    'Default City',
    NULL,
    '000000',
    'Default Country'
)
RETURNING orderid INTO v_order_id;

-- loop through each item in JSON array
FOR v_item IN SELECT * FROM json_array_elements(p_items)

```


LOOP

```
v_product_id := (v_item->>'product_id')::INT;  
v_quantity  := (v_item->>'quantity')::SMALLINT;
```

-- validate product and fetch stock + price

```
SELECT unitsinstock, unitprice  
INTO v_stock, v_price  
FROM products  
WHERE productid = v_product_id  
FOR UPDATE;
```

IF NOT FOUND THEN

```
    RAISE EXCEPTION 'product % does not exist', v_product_id;  
END IF;
```

-- validate quantity

```
IF v_quantity <= 0 THEN  
    RAISE EXCEPTION 'invalid quantity for product %', v_product_id;  
END IF;
```

-- check stock

```
IF v_stock < v_quantity THEN  
    RAISE EXCEPTION 'insufficient stock for product % (available: %, requested: %)',  
        v_product_id, v_stock, v_quantity;  
END IF;
```

-- insert order_details

```
INSERT INTO order_details (  
    orderid,  
    productid,  
    unitprice,  
    quantity,  
    discount  
)  
VALUES (  
    v_order_id,  
    v_product_id,  
    v_price,  
    v_quantity,  
    0  
);
```

-- update stock

```
UPDATE products
```

```
SET unitsinstock = unitsinstock - v_quantity  
WHERE productid = v_product_id;
```

```
END LOOP;
```

```
END;  
$$;
```

```
CALL proc_place_multi_product_order(  
  'VINET'::CHAR(5),  
  5,  
  '[{"product_id":1,"quantity":2},  
   {"product_id":2,"quantity":3}]'::JSON  
);
```

Output:

```
CALL
```

```
Query returned successfully in 56 msec.
```